

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    // Create 3 child processes
    for (int i = 1; i <= 3; i++) {
        pid = fork();

        if (pid < 0) {
            printf("Fork failed!\n");
            return 1;
        }

        if (pid == 0) {
            printf("Child %d started. PID = %d, PPID = %d\n",
                i, getpid(), getppid());

            sleep(i * 2);

            printf("Child %d terminated. PID = %d\n", i, getpid());
            exit(i);
        }
    }

    // Parent waits for any child
    for (int i = 0; i < 3; i++) {
        pid_t child_pid = wait(&status);

        printf("\nParent received termination of child PID = %d\n",
            child_pid);

        if (WIFEXITED(status)) {
            printf("Child exit status = %d\n",
                WEXITSTATUS(status));
        }
    }

    printf("\nAll child processes completed.\n");
    printf("Parent process terminating.\n");

    return 0;
}

```

Output:

```
[venkat@Venkats-MacBook-Air ~ % nano week-4.c
[venkat@Venkats-MacBook-Air ~ % gcc week-4.c
[venkat@Venkats-MacBook-Air ~ % ./a.out
Parent Process Started
Parent PID: 75097

Child 1 started. PID = 75098, PPID = 75097
Child 2 started. PID = 75099, PPID = 75097
Child 3 started. PID = 75100, PPID = 75097
Child 1 terminated. PID = 75098

Parent received termination of child PID = 75098
Child exit status = 1
Child 2 terminated. PID = 75099

Parent received termination of child PID = 75099
Child exit status = 2
Child 3 terminated. PID = 75100

Parent received termination of child PID = 75100
Child exit status = 3

All child processes completed.
Parent process terminating.
```